DEVOPS SHACK
LEARN · BUILD · GROW

PEOPLE
PRACTICES
TOOLS
BETTER SYSTEMS

Monitor
Better
Deploy
Safer
Stay Online
Longer

OBSERVE
LEARN
IMPROVE
REPEAT

50 Monitoring Mistakes

That Cause Production Outage

Real Mistakes. Real Outages. Real Lessons.



PRACTICAL INSIGHTS | REAL-WORLD EXAMPLES | ACTIONABLE TAKEAWAYS | BUILD A MORE RESILIENT FUTURE

DEVOPS SHACK
LEARN · BUILD · GROW

DevOps Shack

DEVOPS & SRE FIELD GUIDE

50 Monitoring Mistakes

That Cause Production Outages

A practical, in-depth breakdown of the alerting, metrics, logging, threshold, on-call, tooling, and process failures that turn small problems into major outages — and exactly how to fix each one.

Prepared by DevOps Shack
devopsshack.com | @DevOpsShack
July 2026

© 2026 DevOps Shack

Introduction

Production outages are rarely caused by a single dramatic failure. In almost every postmortem, the technical root cause is accompanied by a second, quieter root cause: a monitoring or alerting gap that meant the team either didn't know the problem was happening, didn't know fast enough, or knew but couldn't act on it effectively.

This guide documents 50 of the most common monitoring mistakes that turn manageable incidents into full-blown production outages, organized into eight categories that mirror how a monitoring stack is actually built: alerting, instrumentation, dashboards, logging, thresholds and SLOs, on-call process, tooling, and organizational culture.

Each mistake is broken down into three parts: what the mistake actually looks like in a real system, why it specifically leads to outages or extends their duration, and a concrete fix you can apply to your own stack. This is written for DevOps engineers, SREs, platform teams, and engineering leaders who want to move from reactive firefighting to a monitoring practice that catches problems before customers do.

© 2026 DevOps Shack

Category 1: Alerting Mistakes

Alerting is the nervous system of production monitoring. When it is designed poorly, teams either miss the outage entirely or drown in noise until they stop trusting alerts altogether. This category covers the most common ways alerting pipelines fail.

Mistake #1: Alerting on Symptoms Instead of Root Causes

The Problem: Teams alert on high-level symptoms such as "API is slow" without any alert tied to the underlying resource, dependency, or component that actually failed. The on-call engineer wakes up to a vague page and has no starting point for investigation.

Why It Causes Outages: Without a causal chain, mean-time-to-diagnose (MTTD) balloons because engineers must manually correlate dashboards during the outage instead of being pointed directly at the failing component.

The Fix: Build a layered alert hierarchy: business-symptom alerts (checkout failing) should link directly to the infrastructure and dependency alerts (DB connection pool exhausted, downstream payment API timing out) that explain them. Every symptom alert should carry a runbook link that narrows the search space.

Mistake #2: No Alert Severity Tiers (Everything Is P1)

The Problem: Every alert pages the on-call engineer at 3 AM regardless of actual business impact, from a single failed cron job to a full regional outage.

Why It Causes Outages: When every alert is treated as critical, engineers develop alert blindness and start dismissing pages reflexively, which means the one alert that truly indicates a production outage gets the same distracted response as a false alarm.

The Fix: Define at least three severity tiers (P1 = page immediately, P2 = ticket + business-hours review, P3 = dashboard-only/informational) and enforce them in the alerting rules engine, not just in documentation.

Mistake #3: Missing Alert Ownership and Runbook Links

The Problem: Alerts fire with a metric name and a threshold but no indication of which team owns the service, what the alert means in plain English, or what steps to take first.

Why It Causes Outages: During an active outage, the minutes spent figuring out who owns the failing component and what the standard mitigation is directly extend downtime and customer impact.

The Fix: Every alert definition should include an owning team label, a one-line human description of business impact, and a direct link to a maintained runbook. Treat missing metadata as a blocking issue in alert rule reviews.

Mistake #4: Copy-Pasted Thresholds Across Unrelated Services

The Problem: A CPU-at-80% or latency-over-500ms threshold gets copied from one service's alert rule to a dozen others during a template rollout, without accounting for each service's actual traffic pattern or hardware profile.

Why It Causes Outages: A threshold that is meaningful for a high-throughput stateless API is meaningless for a batch-processing worker or a low-traffic internal service, resulting in either constant false positives or an alert that never fires when it should.

The Fix: Derive thresholds from each service's own historical baseline (percentile-based, not copy-pasted absolutes) and revisit them whenever the service's traffic profile or architecture changes materially.

© 2026 DevOps Shack

Mistake #5: No De-Duplication During Cascading Failures

The Problem: When one root cause (a database going down, for example) triggers failures across twenty dependent services, the alerting system fires twenty to two hundred individual pages instead of one correlated incident.

Why It Causes Outages: Alert storms overwhelm on-call engineers at the exact moment they need to think clearly, and the flood of pages makes it harder, not easier, to identify the actual root cause.

The Fix: Implement alert correlation and grouping (via tools like Alertmanager's grouping/inhibition rules, or an AIOps correlation layer) so that dependent-service alerts are suppressed or bundled under the root-cause incident.

Mistake #6: Alerts Routed to Email Instead of a Paging Tool

The Problem: Critical production alerts are configured to send an email or a Slack message with no escalation path, assuming someone will happen to be watching that channel.

Why It Causes Outages: Email and passive Slack notifications have no guaranteed delivery, no acknowledgment tracking, and no escalation if the first responder is asleep or unavailable, so outages can run for hours before anyone notices.

The Fix: Route every P1/P2 alert through a dedicated paging system (PagerDuty, Opsgenie, Squadcast, or similar) with phone/SMS escalation and mandatory acknowledgment within a defined SLA.

Mistake #7: Flapping Alerts Left Unresolved

The Problem: A metric oscillates around a threshold, firing and auto-resolving an alert dozens of times within an hour, and no one fixes the underlying instability or the threshold logic.

Why It Causes Outages: Flapping alerts train the team to ignore notifications from that specific check, so when the same alert fires because of a genuine outage, it gets dismissed as "just flapping again."

The Fix: Add hysteresis (separate fire and clear thresholds), a "for" duration requiring the condition to persist before paging, and track flap frequency as its own metric so noisy alerts get prioritized for cleanup.

© 2026 DevOps Shack

Category 2: Metrics and Instrumentation Mistakes

You cannot alert on, or troubleshoot with, a metric you never collected. This category covers the instrumentation gaps that leave teams flying blind during an incident.

Mistake #8: Not Monitoring the Four Golden Signals

The Problem: Teams instrument CPU and memory but skip latency, traffic, error rate, and saturation, the four signals Google's SRE book identifies as the minimum needed to understand a service's health.

Why It Causes Outages: A service can have healthy CPU and memory while error rates spike or latency degrades badly enough to violate customer SLAs, and none of that shows up if only resource metrics are tracked.

The Fix: Instrument every user-facing service with latency (p50/p95/p99), traffic (requests per second), errors (rate and type), and saturation (queue depth, connection pool usage) as a non-negotiable baseline before shipping to production.

Mistake #9: Missing Business-Level Metrics

The Problem: Infrastructure dashboards look perfectly green while checkout completion rate, sign-up conversion, or order processing throughput has silently dropped to zero because of a logic bug or a misconfigured feature flag.

Why It Causes Outages: Infrastructure health does not guarantee business functionality; a service can return 200 OK on every request while the actual business transaction inside it fails.

The Fix: Instrument and alert on business KPIs (orders placed per minute, payment success rate, login success rate) alongside infrastructure metrics, and treat sudden deviations as production incidents even when servers look healthy.

Mistake #10: High-Cardinality Metrics Overloading the TSDB

The Problem: Engineers tag metrics with unbounded labels such as user_id, request_id, or raw URL paths, causing the time-series database to generate millions of unique series.

Why It Causes Outages: Cardinality explosions degrade or crash the monitoring backend itself (Prometheus OOMs, Datadog custom-metric billing spikes), meaning the monitoring system fails at the exact moment it is needed most.

The Fix: Keep label values bounded and enumerable (status_code, region, service_name), push high-cardinality data to logs or traces instead of metrics, and enforce cardinality limits/linting in the metrics pipeline.

Mistake #11: Metrics Collected but Never Graphed or Reviewed

The Problem: Dozens of custom metrics are emitted by application code because "it seemed useful," but no dashboard, alert, or engineer has looked at them since they were added.

Why It Causes Outages: Unused metrics consume storage and ingestion cost while providing zero incident-response value, and their presence creates false confidence that "we're monitoring that."

The Fix: Run a quarterly metrics audit: any metric with zero dashboard references and zero alert usage gets flagged for removal or must be justified and wired into an actual dashboard/alert.

Mistake #12: No Distributed Tracing Across Microservices

© 2026 DevOps Shack

The Problem: A request crosses eight microservices, but there is no trace ID propagation, so when latency spikes, engineers cannot tell which of the eight hops is actually slow.

Why It Causes Outages: Without tracing, root-causing latency or error issues in a distributed system becomes a manual, service-by-service log hunt that can take hours during an active incident.

The Fix: Adopt OpenTelemetry (or a vendor equivalent) for consistent trace-context propagation across all services, and require every new service to emit spans before it is allowed into production.

Mistake #13: Overly Aggressive Sampling Hiding Real Issues

The Problem: To control cost, tracing or logging is sampled at 1% or less, and that sample rate is static regardless of what is happening in the system.

Why It Causes Outages: Rare but severe error conditions, exactly the kind that cause outages, are statistically likely to be excluded by aggressive random sampling, so the data needed to diagnose the outage was never captured.

The Fix: Use error-biased or tail-based sampling that always captures traces containing errors or high latency, while sampling healthy traffic more aggressively to control cost.

Mistake #14: No Synthetic or Black-Box Monitoring

The Problem: Monitoring relies entirely on internal, white-box metrics emitted by the application itself, with nothing simulating an actual external user hitting the login page, checkout flow, or API endpoint from outside the network.

Why It Causes Outages: Internal metrics can look completely healthy while DNS misconfiguration, a bad load balancer rule, a CDN issue, or a TLS certificate expiry makes the service completely unreachable to real users.

The Fix: Run synthetic checks (Pingdom, Checkly, Grafana Synthetic Monitoring, or scripted Playwright/Selenium journeys) against production from outside your network on a schedule of one to five minutes for critical user journeys.

Mistake #15: Real User Monitoring (RUM) Ignored

The Problem: All monitoring is server-side; nobody tracks actual page load times, JS errors, or API call failures as experienced in real users' browsers and devices.

Why It Causes Outages: Server-side metrics can be perfectly healthy while a bad frontend deploy, a third-party script failure, or a CDN edge-node issue causes real users on specific browsers or regions to experience a full outage.

The Fix: Deploy a RUM tool (Datadog RUM, New Relic Browser, Sentry Performance) to capture client-side errors and performance, segmented by browser, device, and geography.

© 2026 DevOps Shack

Category 3: Dashboard and Visualization Mistakes

A dashboard is only useful if the right person can find the right signal in seconds during an incident. This category covers the dashboard anti-patterns that slow down diagnosis.

Mistake #16: Dashboard Sprawl With No Single Source of Truth

The Problem: Every team, and sometimes every engineer, creates their own dashboard for the same service, resulting in dozens of inconsistent, partially-overlapping views with no canonical version.

Why It Causes Outages: During an incident, engineers waste critical minutes figuring out which of fifteen dashboards is current and accurate, and different dashboards showing conflicting numbers erode trust in the data itself.

The Fix: Designate one canonical service-overview dashboard per service, link it from the service catalog and every alert, and archive or clearly label all others as personal/experimental.

Mistake #17: No Deployment Annotations on Dashboards

The Problem: Metrics graphs show a spike or a drop with no marker indicating that a deployment, feature flag change, or config push happened at that exact time.

Why It Causes Outages: Correlating an incident with its actual trigger (a bad deploy) takes far longer when engineers have to cross-reference a separate deployment log against the metrics timeline by hand.

The Fix: Pipe deploy events, config changes, and feature-flag toggles into the monitoring tool as annotations (Grafana annotations, Datadog events) so every dashboard shows exactly when a change happened.

Mistake #18: Vanity Metrics Dashboards Instead of Actionable Ones

The Problem: Dashboards are built around impressive-looking aggregate numbers (total requests served, total uptime percentage) rather than the specific, actionable signals an on-call engineer needs during an incident.

Why It Causes Outages: Vanity metrics answer "how are we doing overall" for leadership, not "what is broken right now" for the responder, so they add no diagnostic value during an actual outage.

The Fix: Separate executive/vanity dashboards from operational dashboards explicitly, and design operational dashboards around the golden signals and known failure modes of the specific service.

Mistake #19: No Mobile or On-Call-Friendly Dashboard View

The Problem: Dashboards are built exclusively for a wide desktop monitor with dozens of small panels, and the same view is unreadable on the phone an on-call engineer is holding at 2 AM.

Why It Causes Outages: The engineer who gets paged away from their desk cannot quickly assess severity or start diagnosis from their phone, adding real minutes to response time for every off-hours incident.

The Fix: Maintain a condensed, mobile-friendly "first-look" dashboard with only the golden signals and top-line health checks, separate from the deep-dive desktop dashboard.

Mistake #20: Dashboards Not Updated After Architecture Changes

The Problem: A service migrates from a monolith to microservices, or from EC2 to Kubernetes, and the old dashboards keep querying metrics sources or hosts that no longer exist or are no longer representative.

Why It Causes Outages: Stale dashboards silently show empty graphs or misleading old data, and an engineer trusting them during an incident is effectively looking at the wrong system entirely.

© 2026 DevOps Shack

The Fix: Make dashboard updates a mandatory checklist item in the architecture-change or migration runbook, with a review step before the old infrastructure is decommissioned.

Mistake #21: No Correlation Between Infrastructure and Application Dashboards

The Problem: Infrastructure metrics (node CPU, disk IO, network) live in one tool and application metrics (request latency, error rate) live in a completely separate tool with no shared timeline or linking.

Why It Causes Outages: Diagnosing whether an application slowdown was caused by noisy-neighbor CPU contention, disk saturation, or a network issue requires manually toggling between disconnected tools and eyeballing timestamps.

The Fix: Standardize on a unified observability platform, or at minimum ensure both layers use synchronized time ranges and cross-links, so infra and app views can be examined side by side.

© 2026 DevOps Shack

Category 4: Logging Mistakes

Logs are usually the last line of defense once metrics have told you something is wrong. Poor logging practices turn that last line of defense into a dead end.

Mistake #22: Logs Not Centralized or Aggregated

The Problem: Application logs sit on individual hosts or containers with no shipping to a centralized platform (ELK, Loki, Splunk, CloudWatch Logs).

Why It Causes Outages: When a container is auto-scaled away or a host is terminated, its logs disappear with it, permanently destroying the exact evidence needed to diagnose the incident that just occurred on it.

The Fix: Ship every log stream to a centralized, durable log platform as a standard part of the base container image or host provisioning, with retention long enough to cover typical incident investigation windows.

Mistake #23: No Structured Logging (Plain Text Logs)

The Problem: Logs are free-form text strings like "Error processing request for user" with no consistent fields, making them impossible to reliably filter, parse, or query at scale.

Why It Causes Outages: During an incident, engineers cannot filter plain-text logs by service, request ID, or error type fast enough, and every log format inconsistency across services multiplies the manual effort.

The Fix: Standardize on structured JSON logging with a consistent schema (timestamp, service, level, trace_id, message, context fields) enforced across all services via a shared logging library.

Mistake #24: Excessive Log Verbosity Drowning the Signal

The Problem: DEBUG-level logging is left enabled in production, generating enormous volumes of noise that bury the handful of ERROR and WARN lines that actually matter.

Why It Causes Outages: High-volume noisy logs make searches slow and expensive, and engineers scanning results during an incident can genuinely miss the one relevant error line among thousands of irrelevant ones.

The Fix: Default production logging to INFO/WARN/ERROR, make DEBUG toggleable at runtime per-service without a redeploy, and route high-volume low-value logs to cheaper cold storage with a shorter hot-tier retention.

Mistake #25: No Log Retention or Rotation Strategy

The Problem: Log files grow unbounded on disk with no rotation policy, or a centralized log platform has no retention/archival tiering configured.

Why It Causes Outages: Unrotated local logs are a classic, entirely preventable cause of production outages: disks fill to 100%, and the application or even the OS itself crashes or refuses new writes.

The Fix: Enforce log rotation (logrotate, or container log driver limits) on every host/container as part of the base image, and configure retention/archival tiers on the centralized platform to control both disk usage and cost.

Mistake #26: Missing Correlation IDs Across Services

The Problem: A single user request that touches five microservices produces five sets of logs with no shared identifier linking them together.

Why It Causes Outages: Without a correlation or trace ID threaded through every hop, reconstructing the full lifecycle of one failed request means manually guessing at timestamps across five separate log streams.

© 2026 DevOps Shack

The Fix: Generate a correlation/trace ID at the edge (API gateway or load balancer) and propagate it through every downstream call and log line, ideally aligned with your distributed tracing IDs.

Mistake #27: Sensitive Data Logged, Forcing After-the-Fact Redaction

The Problem: PII, credentials, or payment data get logged in plain text, and the fix, once discovered, is to aggressively strip or redact entire log categories in a panic.

Why It Causes Outages: Overcorrecting redaction after a compliance incident often removes fields that were also operationally useful, leaving the team with less diagnostic visibility than before, at the same time compliance risk from the original leak still has to be dealt with.

The Fix: Build field-level redaction and PII scrubbing into the logging library from day one, with an explicit allow-list of safe fields, so compliance and observability are never in conflict.

© 2026 DevOps Shack

Category 5: Threshold, SLO, and Baseline Mistakes

Even perfect metrics and logs are useless if the thresholds and targets built on top of them are wrong. This category covers the mistakes teams make in defining what "good" and "bad" actually mean.

Mistake #28: Static Thresholds Instead of Dynamic Baselines

The Problem: Alert thresholds are hardcoded flat numbers (e.g., latency > 300ms) that never adapt to time-of-day, day-of-week, or seasonal traffic patterns.

Why It Causes Outages: A fixed threshold is either too sensitive during low-traffic periods (causing false alarms) or too lax during high-traffic peaks (missing real degradation), because normal behavior itself changes throughout the day.

The Fix: Use anomaly-detection or dynamic-baseline alerting (available in most modern APM/observability tools) that compares current behavior to the historical pattern for that specific hour and day, layered on top of, not instead of, hard safety-limit thresholds.

Mistake #29: No SLIs or SLOs Defined at All

The Problem: Teams monitor dozens of raw metrics but have never formally defined which ones are Service Level Indicators, or set a Service Level Objective and error budget against them.

Why It Causes Outages: Without an SLO, there is no shared, objective definition of "the service is currently broken," so incident declaration becomes a subjective judgment call that different engineers make differently, delaying response.

The Fix: Define two to five SLIs per critical service (availability, latency, error rate) with explicit SLO targets and error budgets, and tie your P1 alerting directly to error-budget burn rate, not just raw thresholds.

Mistake #30: Alert Thresholds Set Once and Never Revisited

The Problem: A threshold was tuned correctly at launch two years ago and has never been revisited even though traffic has grown 10x and the architecture has changed multiple times.

Why It Causes Outages: Thresholds that were once well-calibrated silently drift out of relevance as the system evolves, becoming either constant false-positive noise or dangerously insensitive to real problems.

The Fix: Schedule a recurring quarterly alert-threshold review as a standing engineering ritual, and trigger an ad hoc review any time a service's traffic or architecture changes materially.

Mistake #31: Ignoring Seasonality and Batch/Cron Patterns

The Problem: Nightly batch jobs, month-end billing runs, or Black-Friday-style traffic spikes are treated identically to normal traffic by the alerting rules, instead of being modeled as expected patterns.

Why It Causes Outages: Predictable, recurring load patterns generate recurring false alarms if unaccounted for, which trains the team to ignore alerts during exactly the windows when a real problem is also most likely to occur.

The Fix: Build known seasonal and batch-job patterns into alerting logic with time-window-aware thresholds or scheduled alert silencing, documented so the exception is visible and intentional, not a workaround.

Mistake #32: No Error Budget Policy Tied to Alerting or Release Decisions

© 2026 DevOps Shack

The Problem: Even where an SLO exists on paper, there is no actual policy connecting error-budget consumption to concrete actions like halting risky deploys or triggering a review.

Why It Causes Outages: An SLO with no enforcement mechanism is just a number on a dashboard; teams keep shipping risky changes even after the error budget is already exhausted, directly increasing the odds of the next outage.

The Fix: Define and enforce an error budget policy: when the budget is burned past an agreed threshold, non-essential releases pause and the team prioritizes reliability work until the budget recovers.

Mistake #33: Capacity and Saturation Metrics Ignored Until Too Late

The Problem: Disk usage, connection pool utilization, queue depth, and thread pool saturation are collected but have no alerting attached, so the team only notices when the resource is already exhausted.

Why It Causes Outages: Saturation-type failures (disk full, connection pool exhausted, queue backed up) are almost always predictable well in advance from a slow, steady trend line, yet they are one of the most common causes of complete outages because no one is watching the trend.

The Fix: Alert on saturation trend and forecasted time-to-exhaustion (e.g., "disk will be full in 6 hours at current growth rate"), not just on the resource already being at 100%.

© 2026 DevOps Shack

Category 6: On-Call and Incident Response Mistakes

Monitoring only pays off if the humans behind it can respond effectively. This category covers the process failures that turn a detected issue into a prolonged outage.

Mistake #34: No Documented On-Call Rotation or Escalation Policy

The Problem: On-call responsibility is informal, tribal knowledge, with no written schedule, no documented escalation path, and no clarity on who is secondary if the primary doesn't respond.

Why It Causes Outages: During a real outage, ambiguity about who is responsible for responding costs minutes to hours, especially if the primary on-call engineer is unreachable and no automatic escalation exists.

The Fix: Codify the on-call rotation and escalation policy in the paging tool itself (not a wiki page), with automatic escalation to secondary and then to a manager after a defined no-response window.

Mistake #35: Alert Fatigue From Unactionable Pages

The Problem: The on-call engineer receives frequent pages for conditions that require no immediate action, self-resolve, or are simply informational.

Why It Causes Outages: Chronic alert fatigue is one of the strongest predictors of missed real incidents, because human responders physiologically and psychologically desensitize to frequent low-value interruptions.

The Fix: Track pages-per-on-call-shift as a first-class engineering metric, set a target ceiling, and treat exceeding it as a mandatory trigger for an alert-tuning sprint.

Mistake #36: No Postmortem Process, So Outages Repeat

The Problem: After an incident is resolved, the team moves on immediately with no blameless postmortem, no documented root cause, and no tracked follow-up actions.

Why It Causes Outages: Without a postmortem, the same class of outage recurs because the systemic gap that allowed it (a missing alert, a bad threshold, a single point of failure) was never actually fixed, only the symptom was.

The Fix: Require a blameless postmortem with a documented timeline, root cause, and tracked action items for every P1/P2 incident, and review open action items in a recurring reliability meeting until closed.

Mistake #37: Missing Runbooks for Known Failure Modes

The Problem: Common, previously-seen failure modes (a specific service running out of connections, a known third-party API rate limit) have no documented runbook, so each occurrence is re-diagnosed from scratch.

Why It Causes Outages: Re-diagnosing a previously-solved problem wastes time during every recurrence and puts unnecessary pressure on the few engineers who happen to remember how it was fixed last time.

The Fix: Write a runbook for every incident during its postmortem, link it directly from the corresponding alert, and periodically test that runbooks are still accurate as the system evolves.

Mistake #38: No Incident Communication Channel or Status Page

The Problem: During an outage, there is no designated communication channel for internal stakeholders and no public status page for customers, so information trickles out inconsistently or not at all.

© 2026 DevOps Shack

Why It Causes Outages: Poor incident communication compounds the business damage of an outage independently of its technical severity, eroding customer trust and creating internal chaos as multiple people ask "what's happening" simultaneously.

The Fix: Establish a standard incident-communication protocol (dedicated Slack/Teams channel, an Incident Commander role, and a public status page like Statuspage or Cachet) that activates automatically when a P1 is declared.

Mistake #39: Single Point of Knowledge (Bus Factor of One)

The Problem: Only one engineer truly understands how a critical system's monitoring and alerting is configured, and that person is unavailable during the incident.

Why It Causes Outages: When the one person who understands a system's quirks is unreachable, the rest of the team loses hours re-learning what that person already knew, directly extending the outage.

The Fix: Mandate that monitoring configuration, dashboards, and runbooks are documented and reviewed by at least one other engineer, and rotate on-call/ownership deliberately to spread knowledge across the team.

Mistake #40: No Chaos Engineering to Test Monitoring Under Failure

The Problem: Monitoring and alerting have never been deliberately tested by injecting a real failure; the team only discovers gaps when an actual, unplanned outage happens.

Why It Causes Outages: Monitoring gaps (a missing alert, a broken dashboard query, a paging integration that silently stopped working) are invisible until the moment they are needed, which is exactly the worst time to discover them.

The Fix: Run regular game days or chaos engineering exercises (using tools like Chaos Mesh, Gremlin, or manual failure injection) that deliberately break something and verify the correct alerts fire, page the right people, and the runbook works.

© 2026 DevOps Shack

Category 7: Tooling and Infrastructure Mistakes

The monitoring stack is itself a piece of production infrastructure, and it fails in the same ways any other system does if it is not treated with the same rigor.

Mistake #41: Monitoring Stack Itself Is a Single Point of Failure

The Problem: The Prometheus server, Grafana instance, or log aggregator runs as a single unreplicated node with no redundancy.

Why It Causes Outages: The worst possible time for the monitoring system to go down is during the exact infrastructure event (a host failure, a power event, a network partition) that monitoring exists to detect, and an unreplicated monitoring stack is highly likely to fail alongside it.

The Fix: Run the monitoring stack in a highly available configuration (replicated Prometheus with Thanos/Cortex/Mimir, clustered Elasticsearch, multi-node Grafana) with the same reliability standards applied to production services.

Mistake #42: Monitoring Deployed in the Same Failure Domain as Production

The Problem: The monitoring and alerting infrastructure lives in the exact same region, availability zone, or network segment as the production systems it watches.

Why It Causes Outages: A full regional outage takes down both the production service and the monitoring system watching it simultaneously, meaning the team has zero visibility during precisely the largest-impact incidents.

The Fix: Deploy monitoring, alerting, and the paging pipeline in a separate region or provider from primary production infrastructure so a regional failure does not blind the team at the same moment it needs visibility most.

Mistake #43: No Meta-Monitoring of the Monitoring System

The Problem: Nothing monitors whether Prometheus scrape jobs are succeeding, whether the alerting pipeline is actually delivering pages, or whether the log shipper is still running.

Why It Causes Outages: A silently broken exporter, a stuck alert-manager, or a dead log-shipping agent produces no visible error, just a slow, invisible loss of coverage that is usually discovered only during a real incident when the expected alert never fires.

The Fix: Implement a "dead man's switch" heartbeat alert that pages the team if the monitoring pipeline itself stops reporting, and monitor scrape/ingest success rates as their own first-class metric.

Mistake #44: Agent or Exporter Misconfiguration Silently Dropping Metrics

The Problem: A metrics agent or exporter is misconfigured, rate-limited, or crash-looping, and metrics simply stop arriving with no obvious error surfaced anywhere.

Why It Causes Outages: A dashboard with a flat, empty line looks calm rather than alarming, so engineers frequently mistake "no data" for "everything is fine" instead of recognizing it as a coverage gap.

The Fix: Alert explicitly on the absence of expected data (a metric that should always be reporting but has gone silent for N minutes), not only on threshold breaches within the data that does arrive.

Mistake #45: No Monitoring Coverage for Third-Party Dependencies

© 2026 DevOps Shack

The Problem: External APIs, payment gateways, SaaS integrations, and managed cloud services that the application depends on have no monitoring of their own, so failures there look identical to an internal application error.

Why It Causes Outages: When a third-party dependency degrades or goes down, engineers waste critical time investigating their own code before realizing the actual failure is outside their control, and they have no independent visibility into that dependency's status.

The Fix: Subscribe to and integrate third-party status pages/APIs into your own monitoring, add explicit synthetic checks against critical external dependencies, and instrument dependency-specific error rates and latency separately from your own service's.

Mistake #46: Cost-Driven Sampling That Blinds Visibility During Incidents

The Problem: To control observability vendor bills, ingestion is capped or sampled down aggressively across the board, including exactly the high-cardinality error data that matters most during an incident.

Why It Causes Outages: Cost optimization applied uniformly, rather than intelligently, means the data most needed during the 1% of the time an outage is happening is the same data most likely to have been dropped to save money during the other 99%.

The Fix: Apply cost controls selectively: sample healthy, low-value traffic aggressively while always fully capturing error, warning, and anomalous data, and budget observability cost as a fraction of the cost of the outages it prevents, not as a pure line-item to minimize.

© 2026 DevOps Shack

Category 8: Process and Culture Mistakes

The final and most persistent category of monitoring mistakes has nothing to do with tools at all, and everything to do with how an organization treats observability as a discipline.

Mistake #47: Monitoring Treated as an Afterthought, Added Post-Incident

The Problem: New services ship to production with zero dashboards, zero alerts, and zero SLOs, and monitoring only gets bolted on reactively after the service has already caused an outage.

Why It Causes Outages: Reactive monitoring guarantees that every service experiences at least one "free" undetected outage before anyone builds visibility into it, and that pattern repeats for every new service the organization ships.

The Fix: Make monitoring, alerting, and a service-overview dashboard a mandatory, blocking part of the production-readiness checklist before any service can be deployed to production traffic.

Mistake #48: No Clear Ownership Model ("Someone Else's Dashboard")

The Problem: Dashboards and alerts exist for a service, but no specific team or engineer is accountable for keeping them accurate, relevant, or tuned over time.

Why It Causes Outages: Without a named owner, monitoring quality decays silently: thresholds go stale, dashboards break after refactors, and nobody notices because nobody feels responsible for noticing.

The Fix: Assign explicit monitoring ownership per service as part of the service catalog/ownership model, with that ownership reviewed alongside general service ownership during team reorganizations.

Mistake #49: Monitoring Not Part of CI/CD or Definition of Done

The Problem: A feature or service is considered "done" once the code merges and tests pass, with no requirement that corresponding metrics, alerts, or dashboard updates ship alongside it.

Why It Causes Outages: Features shipped without matching observability updates create permanent blind spots the moment they go live, and those gaps compound as more features ship the same way over time.

The Fix: Add an observability checklist item to the pull-request template and Definition of Done: new metrics instrumented, alerts updated if behavior changed, and dashboard panels added for any new critical path.

Mistake #50: No Regular Alert Reviews or Noise Pruning

The Problem: Alert rules accumulate over years with no process to periodically review which ones still fire usefully versus which have become permanent noise that gets muted or ignored.

Why It Causes Outages: Accumulated alert cruft is the single biggest driver of long-term alert fatigue, and teams that never prune it eventually reach a state where on-call engineers mute entire alert categories just to get sleep, silencing real signals along with the noise.

The Fix: Run a recurring (monthly or quarterly) alert review that examines fire frequency, false-positive rate, and time-to-acknowledge for every alert rule, and retires or retunes anything that consistently fails to indicate actionable, real problems.

© 2026 DevOps Shack

Quick Reference Checklist

Use this table as a scannable audit checklist against your own stack — walk through each row and mark whether your team has this mistake covered.

#	Category	Mistake
1	Alerting Mistakes	Alerting on Symptoms Instead of Root Causes
2	Alerting Mistakes	No Alert Severity Tiers (Everything Is P1)
3	Alerting Mistakes	Missing Alert Ownership and Runbook Links
4	Alerting Mistakes	Copy-Pasted Thresholds Across Unrelated Services
5	Alerting Mistakes	No De-Duplication During Cascading Failures
6	Alerting Mistakes	Alerts Routed to Email Instead of a Paging Tool
7	Alerting Mistakes	Flapping Alerts Left Unresolved
8	Metrics and Instrumentation Mistakes	Not Monitoring the Four Golden Signals
9	Metrics and Instrumentation Mistakes	Missing Business-Level Metrics
10	Metrics and Instrumentation Mistakes	High-Cardinality Metrics Overloading the TSDB
11	Metrics and Instrumentation Mistakes	Metrics Collected but Never Graphed or Reviewed
12	Metrics and Instrumentation Mistakes	No Distributed Tracing Across Microservices
13	Metrics and Instrumentation Mistakes	Overly Aggressive Sampling Hiding Real Issues
14	Metrics and Instrumentation Mistakes	No Synthetic or Black-Box Monitoring
15	Metrics and Instrumentation Mistakes	Real User Monitoring (RUM) Ignored
16	Dashboard and Visualization Mistakes	Dashboard Sprawl With No Single Source of Truth
17	Dashboard and Visualization Mistakes	No Deployment Annotations on Dashboards
18	Dashboard and Visualization Mistakes	Vanity Metrics Dashboards Instead of Actionable Ones
19	Dashboard and Visualization Mistakes	No Mobile or On-Call-Friendly Dashboard View
20	Dashboard and Visualization Mistakes	Dashboards Not Updated After Architecture Changes
21	Dashboard and Visualization Mistakes	No Correlation Between Infrastructure and Application Dashboards
22	Logging Mistakes	Logs Not Centralized or Aggregated
23	Logging Mistakes	No Structured Logging (Plain Text Logs)
24	Logging Mistakes	Excessive Log Verbosity Drowning the Signal
25	Logging Mistakes	No Log Retention or Rotation Strategy
26	Logging Mistakes	Missing Correlation IDs Across Services
27	Logging Mistakes	Sensitive Data Logged, Forcing After-the-Fact Redaction
28	Threshold, SLO, and Baseline Mistakes	Static Thresholds Instead of Dynamic Baselines
29	Threshold, SLO, and Baseline Mistakes	No SLIs or SLOs Defined at All
30	Threshold, SLO, and Baseline Mistakes	Alert Thresholds Set Once and Never Revisited
31	Threshold, SLO, and Baseline Mistakes	Ignoring Seasonality and Batch/Cron Patterns
32	Threshold, SLO, and Baseline Mistakes	No Error Budget Policy Tied to Alerting or Release Decisions
33	Threshold, SLO, and Baseline Mistakes	Capacity and Saturation Metrics Ignored Until Too Late
34	On-Call and Incident Response Mistakes	No Documented On-Call Rotation or Escalation Policy

© 2026 DevOps Shack

#	Category	Mistake
35	On-Call and Incident Response Mistakes	Alert Fatigue From Unactionable Pages
36	On-Call and Incident Response Mistakes	No Postmortem Process, So Outages Repeat
37	On-Call and Incident Response Mistakes	Missing Runbooks for Known Failure Modes
38	On-Call and Incident Response Mistakes	No Incident Communication Channel or Status Page
39	On-Call and Incident Response Mistakes	Single Point of Knowledge (Bus Factor of One)
40	On-Call and Incident Response Mistakes	No Chaos Engineering to Test Monitoring Under Failure
41	Tooling and Infrastructure Mistakes	Monitoring Stack Itself Is a Single Point of Failure
42	Tooling and Infrastructure Mistakes	Monitoring Deployed in the Same Failure Domain as Production
43	Tooling and Infrastructure Mistakes	No Meta-Monitoring of the Monitoring System
44	Tooling and Infrastructure Mistakes	Agent or Exporter Misconfiguration Silently Dropping Metrics
45	Tooling and Infrastructure Mistakes	No Monitoring Coverage for Third-Party Dependencies
46	Tooling and Infrastructure Mistakes	Cost-Driven Sampling That Blinds Visibility During Incidents
47	Process and Culture Mistakes	Monitoring Treated as an Afterthought, Added Post-Incident
48	Process and Culture Mistakes	No Clear Ownership Model ("Someone Else's Dashboard")
49	Process and Culture Mistakes	Monitoring Not Part of CI/CD or Definition of Done
50	Process and Culture Mistakes	No Regular Alert Reviews or Noise Pruning

© 2026 DevOps Shack

Closing Notes

No monitoring stack gets all 50 of these right on the first attempt — and it doesn't need to. The highest-leverage move is to treat this list as a living audit: pick the five mistakes most likely to be true of your systems today, fix those first, and revisit the rest on a quarterly cadence.

Good monitoring is not a tool you buy once. It is a practice of continuously asking "what would we miss right now, and what would that cost us?" — and closing that gap a little more every sprint.

DevOps Shack
DevOps & Cloud Engineering Education
devopsshack.com | @DevOpsShack

© 2026 DevOps Shack